

05 — Frontend API Integration

Product: Endpoint IQ **Document:** Frontend API Integration

1. API Base URL

The frontend communicates exclusively with the **API Gateway** — the “single front door” per §4.2 of the architecture. It never calls the Auth Service, Spec Import Service, Orchestrator, AI Engine, or Report Service directly.

```
VITE_API_BASE_URL = https://<gateway-host>/api
VITE_WS_BASE_URL  = wss://<gateway-host>/ws
```

Open Decision: Exact base path/versioning scheme (e.g. /api/v1) is not defined in the source document. Recommend versioning from day one (/api/v1) as a standard practice; confirm with backend.

2. HTTP Client

A single shared client instance (Axios recommended for interceptor ergonomics, though fetch is equally valid) configured once in `services/httpClient.ts` and reused by every service module.

```
// services/httpClient.ts
import axios from "axios";

export const httpClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  withCredentials: true, // required: HttpOnly session cookie must be sent
  timeout: 15000,
});
```

`withCredentials: true` is required because the architecture mandates `HttpOnly` cookie-based token storage (§4.3, §6) — the frontend never attaches an `Authorization` header with a token it holds itself for the *target-API* session; that’s managed server-side. Platform-login auth (see Open Decision in 02-...) may add its own cookie the same way.

3. Request Configuration

- All requests default to `Content-Type: application/json` except file downloads.
- Every request includes credentials (cookies) automatically via `withCredentials`.
- Request timeout: 15s default for standard CRUD calls; test-run start/status calls may need a longer timeout or should be fire-and-forget with WS/polling follow-up (long-running operations should never block on a single long HTTP request).

4. Authentication

Two authentication contexts exist in this product — do not conflate them:

Context	Who	Mechanism	Frontend responsibility
Platform auth	The human using Endpoint IQ	Not defined in source — Open Decision (see 02-Frontend-Architecture.md)	Send login form, rely on resulting HttpOnly cookie, redirect on 401
Target-API auth	Endpoint IQ testing <i>your</i> API as Admin/Client/Company	Auth Service calls target's POST /login, stores JWT in Redis session store, HttpOnly, auto-refreshed (§4.3)	Frontend only submits credentials once via the Credentials page (§05 of this doc's endpoint table); it never sees or stores the resulting JWT

5. Headers

- Content-Type: application/json for standard requests.
- No manual Authorization header set by the frontend for target-API testing flows — cookies handle it.
- If platform auth uses a bearer scheme instead of cookies (to be confirmed — Open Decision), a request interceptor would attach it; **do not implement this speculatively** until confirmed, to avoid contradicting the “never in localStorage” rule if the token would need client-side storage.

6. Request Interceptors

- Attach a request ID / correlation header for tracing (aligns with the architecture's observability stack — Prometheus/Jaeger, §6 Observability) — **Proposed** for frontend-side tracing correlation.
- No token injection needed (cookie-based).

7. Response Interceptors

- Normalize all errors into a single shape before they reach UI code:

```
interface ApiError {
  kind: "network" | "timeout" | "auth" | "validation" | "server" | "unknown";
  status?: number;
  message: string;
  details?: Record<string, string[]>; // field-level validation errors
}
```

- On 401, redirect to /login (platform auth) — but distinguish this from a 401/403 that represents a **target-API auth failure surfaced as test data** (e.g. a role's credentials are invalid and every test for that role fails) — the latter is a **test result**, not a frontend auth error, and must never trigger a redirect. See §9 below.

8. Error Handling

Error type	HTTP signal	Frontend behavior
Network error	No response / connection failure	Toast + retry affordance
Timeout	Request exceeds timeout	Toast + retry affordance
Auth failure (platform)	401 from Gateway on a platform-auth-required route	Redirect to /login
Validation error	400/422 with field details	Inline form field errors
Server error	5xx	Generic error banner + “try again”
Bad OpenAPI spec	4xx from POST /import with parse error details	Clear, specific message on the import form (per §10 Risks in the architecture)

9. Authentication Failures (Target API vs. Platform — Critical Distinction)

Because Workers attach role JWTs to outbound test calls, a 401/403 a *test* receives from the target API is **expected test data**, recorded in `test_results`, and must be rendered as a failed/warning result in the Results UI — not treated as an Endpoint IQ frontend auth error. Only 401/403 responses **from the API Gateway itself**, on Endpoint IQ’s own endpoints, should trigger the platform login redirect described in §8.

10. Validation Errors

Field-level errors returned as `{ field: string[] }` are mapped directly onto form fields (Project creation, Role, Credential forms). Client-side validation (required fields, URL format for Swagger import) runs first as a fast-fail UX layer; the server response remains authoritative.

11. Server Errors

Generic 5xx responses show a non-technical error banner with a “Try Again” action and, in non-production environments, the raw error for debugging.

12. Network Errors

Detected via request failure with no response object; shown as “Can’t reach Endpoint IQ — check your connection” with retry.

13. Timeout Handling

Standard CRUD calls: 15s timeout → treated as a server error. Long-running actions (spec import on a very large API, test run start) should be **asynchronous**: the initiating call returns quickly with a job/run ID, and the frontend polls or subscribes via WS for completion rather than holding one long HTTP request open.

14. File Downloads

Report download (GET `/projects/:projectId/reports/:reportId/download` or the `file_url` from the reports record) is handled as a direct browser navigation/`<a download>` or a blob-fetch-then-save, depending on whether `file_url` is a signed direct link or requires an authenticated fetch through the Gateway.

Open Decision: Whether `reports.file_url` is a publicly-signed URL (direct download) or a Gateway-proxied, cookie-authenticated endpoint is not specified in the source data model. Recommend treating it as **Proposed API Contract:** `GET /projects/:projectId/reports/:reportId/download` returning the file through the Gateway (so auth is consistently cookie-based), unless backend confirms a signed-URL approach.

15. Pagination

Not explicitly defined in the source document for any resource. Given that endpoints, roles, and test results can grow large for bigger APIs, pagination (or at minimum cursor/offset support) is recommended.

Open Decision: Pagination scheme (offset/limit vs. cursor) for `GET /projects/:projectId/endpoints` and `GET .../results` is not defined in source. Recommend standard `?page=&pageSize=` query params as a **Proposed API Contract**, confirm with backend before implementation.

16. Real-Time Test Progress

The architecture diagram explicitly shows a REST / WS connection between the Frontend and API Gateway, and the Orchestrator is described as tracking progress and marking a run complete when all jobs finish (§4.5, §3.3). This supports a **WebSocket-based live progress** channel.

Recommended pattern:

```
// features/test-runs/hooks/useTestRunProgress.ts
// Subscribes to ws://.../test-runs/:runId for live job-completion events;
// falls back to polling GET /projects/:projectId/test-runs/:runId every 3-5s
// if the WS connection fails, so the UI degrades gracefully.
```

Open Decision: The exact WS message schema/event names are not defined in source (only that a WS channel exists per the architecture diagram). Treat the message contract below as **Proposed API Contract**:

```
{ "type": "job_completed", "runId": "...", "completed": 42, "total": 120, "lastEndpoint": "...", "status": "completed" }
{ "type": "run_completed", "runId": "...", "status": "completed" }
```

17. API Service Modules

Service module	Responsibility
authService	Platform login/logout (Open Decision, Proposed)
projectService	CRUD for projects
specImportService	Trigger and check status of OpenAPI import
endpointService	List/read discovered endpoints
roleService	CRUD for roles
credentialService	Create/update credentials (write-only)
testRunService	Start/list/get test runs; subscribe to progress
resultService	List/read test results per run, including failed-endpoint detail
aiAnalysisService	Read AI analysis for a run/result
reportService	Read report metadata; trigger download

Service module	Responsibility
----------------	----------------

18. Endpoint Reference

Endpoints explicitly named in the source document are marked **Confirmed**. All others are marked **Proposed API Contract** and must be validated with the backend team before or during implementation — they are included so implementation isn't blocked, not because they are guaranteed final.

Spec Import — Confirmed

Method	POST
Endpoint	/import
Purpose	Import an OpenAPI/Swagger spec by URL; triggers Spec Import Service (§3.1, §4.4)
Request	{ "projectId": string, "swaggerUrl": string }
Response	{ "importId": string, "status": "processing" \ "completed" \ "failed", "endpointCount"? : number }
Errors	400 invalid URL, 422 unparseable spec
Frontend usage	specImportService.importSpec() — called from /projects/new and re-import action on /projects/:projectId/endpoints

Target-API Login (per role) — Confirmed at architecture level, contract inferred

Method	POST (internal — Auth Service calls the <i>target</i> API's own login endpoint; the frontend's job is only to submit credentials to Endpoint IQ, not call this directly)
Endpoint	Frontend-facing: /projects/:projectId/credentials (Proposed shape below)
Purpose	Store role credentials so the Auth Service can log in as that role (§4.3)
Request	{ "roleId": string, "username": string, "password": string }
Response	{ "credentialId": string, "status": "saved" }
Errors	400 validation, 409 credential already exists (if update-only via PUT)
Frontend usage	credentialService.saveCredential()

Marked **Proposed API Contract** for the exact request/response shape — the source confirms this data flow (§3.2, §5 credentials table) but not the literal frontend REST contract.

All Remaining Endpoints — Proposed API Contract

The source document confirms the underlying services and data model but does not enumerate REST paths for project, role, endpoint, test-run, result, AI-analysis, or report retrieval. The table below proposes a consistent, RESTful contract derived directly from the data model (§5) and workflow (§3), to unblock frontend implementation. **All rows in this table require backend confirmation.**

Method	Endpoint	Purpose
POST	/projects	Create project
GET	/projects	List projects
GET	/projects/:projectId	Get project detail
DELETE	/projects/:projectId	Delete project
GET	/projects/:projectId/endpoints	List imported endpoints
GET	/projects/:projectId/roles	List roles
POST	/projects/:projectId/roles	Create role
PUT	/projects/:projectId/roles/:roleId	Update role
DELETE	/projects/:projectId/roles/:roleId	Delete role
POST	/projects/:projectId/test-runs	Start test run
GET	/projects/:projectId/test-runs	List test runs
GET	/projects/:projectId/test-runs/:runId	Get run status/progress
GET	/projects/:projectId/test-runs/:runId/results	List test results for a run
GET	/projects/:projectId/test-runs/:runId/results/:resultId	Get single failed/passed result detail
GET	/projects/:projectId/test-runs/:runId/results/:resultId/ai-analysis	Get AI analysis for a result
GET	/projects/:projectId/reports	Get reports summary
GET	/projects/:projectId/reports/:reportId/download	Download report file

Each frontend service module (§17) should be built against this table, with a clear TODO/comment noting “Proposed API Contract — confirm with backend” until each is verified.

Cross-references: 06-Frontend-State-and-Data-Models.md (request/response type definitions), 07-Frontend-Test-Results-and-AI-Report-UI.md (usage in the results/AI UI).